

ESO-207: Theoretical Assignment #1

Aviral Gupta

230246

Indian Institute of Technology, Kanpur

Question 1: Maximal Product sum

We are given two integer sequences.

$$a = [a_1, a_2, \dots, a_n], \quad b = [b_1, b_2, \dots, b_n],$$

each containing n integers. We need to compute

$$P = \max_{\pi} \sum_{i=1}^n a_i \cdot b_{\pi(i)},$$

where π is a permutation of the set $\{1, 2, \dots, n\}$. This value P represents the maximum possible dot product of the two sequences with rearrangement of the second sequence.

Question (a)

Design an algorithm to determine the value of P and write a neat pseudo-code for it.

Solution:

We know that P is maximised when both a and b are sorted in the same order; therefore, it is optimal to approach sorting a and b in ascending order and then compute the dot product element-wise.

Pseudocode:

Algorithm 1: MaximalDotProduct

Input: Two integer arrays $a[1..n]$ and $b[1..n]$

Output: Maximum possible dot product P

```
1 Sort  $a$  in ascending order;
2 Sort  $b$  in ascending order;
3  $P \leftarrow 0$ ;
4 for  $i \leftarrow 1$  to  $n$  do
5    $P \leftarrow P + (a[i] \cdot b[i])$ ;
6 return  $P$ ;
```

Time Complexity Analysis

- Sorting a : $\Theta(n \log n)$
- Sorting b : $\Theta(n \log n)$
- Dot product computation: $\Theta(n)$

Total Time Complexity: $\Theta(n \log n)$

Question (b)

Now, suppose over time, the sequences a and b are updated through q operations. Each operation is described by two integers (o, x) where:

- $1 \leq x \leq n$
- If $o = 1$, increment a_x by 1
- If $o = 2$, increment b_x by 1

Your task is to design an algorithm to find out the value of P in q different situations. Write neat pseudocode for the problem.

Solution:

Our goal is to compute the *maximum dot product* after each update:

$$P = \max_{\pi} \sum_{i=1}^n a_i \cdot b_{\pi(i)}$$

P is maximised when both arrays a and b are sorted in the same order (ascending). To handle updates efficiently, we maintain both arrays in **sorted order** and **locally fix any disorder** using binary search and repositioning when an element is incremented.

Algorithm Idea

- We maintain both arrays a and b and $\bar{a}$ and $\bar{b}$, their sorted order.
- Each update operation is of the form (o, x) :
 - If $o = 1$, we increment $\bar{a}[i]$ by 1 where i is the last index of a in sorted array with value $a[x]$.
 - If $o = 2$, we increment $\bar{b}[i]$ by 1 where i is the last index of b in sorted array with value $b[x]$.

Strategy When $o = 1$ (Increment $a[x]$)

1. Find index i , the last index of sorted array $\bar{a}$ with value $a[x]$ using binary search.
2. Let $\bar{a}[i] \leftarrow \bar{a}[i] + 1$ and $a[x] \leftarrow a[x] + 1$.
3. As we choose the last index, if we increment the value $a[x]$ by 1, the array will remain sorted as $a[x+1] > a[x]$, so $a[x] + 1 \leq a[x+1]$, so the dot product P changes by:

$$P = P + 1 \cdot \bar{b}[i]$$

All other entries remain unchanged, so no other part of P is affected.

Similarly, we can apply the same logic when $o = 2$ (increment $b[x]$).

1. Find index i , the last index of sorted array $\bar{b}$ with value $b[x]$ using binary search.
2. Let $\bar{b}[i] \leftarrow \bar{b}[i] + 1$ and $b[x] \leftarrow b[x] + 1$.
3. As we choose the last index, if we increment the value $b[x]$ by 1, the array will remain sorted as $b[x+1] > b[x]$, so $b[x] + 1 \leq b[x+1]$, so the dot product P changes by:

$$P = P + 1 \cdot \bar{a}[i]$$

All other entries remain unchanged, so no other part of P is affected.

Pseudocode:**Algorithm 2:** UpdateDotProduct**Input:** Normal sequence $a[1..n]$, $b[1..n]$, Sorted arrays $\bar{a}[1..n]$, $\bar{b}[1..n]$, and q updates (o, x) **Output:** Updated dot product P after each operation

```

1 Compute  $P \leftarrow \sum_{i=1}^n a[i] \cdot b[i]$ ;
2 foreach operation  $(o, x)$  do
3   if  $o = 1$  then
4      $val \leftarrow a[x]$ ;
5     Find last position  $i$  in  $\bar{a}$  such that  $\bar{a}[i] = val$  (binary search);
6      $a[x] \leftarrow a[x] + 1$ ;
7      $\bar{a}[i] \leftarrow \bar{a}[i] + 1$ ;
8      $P \leftarrow P + \bar{b}[i]$ ;
9   else if  $o = 2$  then
10     $val \leftarrow b[x]$ ;
11    Find last position  $i$  in  $\bar{b}$  such that  $\bar{b}[i] = val$  (binary search);
12     $b[x] \leftarrow b[x] + 1$ ;
13     $\bar{b}[i] \leftarrow \bar{b}[i] + 1$ ;
14     $P \leftarrow P + \bar{a}[i]$ ;
15 Print  $P$ ;

```

Question (c)

Give proof of correctness and time complexity analysis of your approach for part (b)

Solution:**Goal:** Maintain the maximum dot product

$$P = \max_{\pi} \sum_{i=1}^n a_i \cdot b_{\pi(i)}$$

after q updates to sequences a and b .**Key Observation:** P is maximised when both arrays are sorted in the same order:

$$P = \sum_{i=1}^n \bar{a}_i \cdot \bar{b}_i$$

where $\bar{a}$ and $\bar{b}$ are the sorted versions of a and b respectively.**Proof of Correctness**

We demonstrate that, after each update, the invariant that both arrays are sorted and the dot product is maximised remains valid.

- **Invariant:** Arrays $\bar{a}$ and $\bar{b}$ remain sorted in non-decreasing order, and P is computed as

$$P = \sum_{i=1}^n \bar{a}_i \cdot \bar{b}_i$$

- **Initialization:** Initially, both arrays are sorted, and P is computed accordingly.
- **Update Operation** ($o = 1$, i.e., increment $a[x]$):

- Let $a[x] = v$. We find the last index i in $\bar{a}$ such that $\bar{a}[i] = v$.

- We increment this value to $v + 1$, as i is the last index with value v so the next element in $\bar{a}$ will have value $> v$ so even after updating the value of $a[x]$, $\bar{a}$ remains sorted so the change is the index i in the sequence and every other element remains same.
- Only one element is changed, and it does not need to be repositioned, so the rest of the array remains unchanged.
- The change in P is only due to the change in the element of $\bar{a}[i]$:

$$\text{Old contribution: } v \cdot \bar{b}[i], \quad \text{New contribution: } (v + 1) \cdot \bar{b}[i]$$

- So, P is updated as:

$$P \leftarrow P - v \cdot \bar{b}[i] + (v + 1) \cdot \bar{b}[i]$$

• **Update Operation ($o = 2$, i.e., increment $b[x]$):**

- Analogously, we update $b[x] = v \rightarrow v + 1$.
- We increment this value to $v + 1$, as i is the last index with value v so the next element in $\bar{b}$ will have value $> v$ so even after updating the value of $b[x]$, $\bar{b}$ remains sorted so the change is the index i in the sequence and every other element remains same.
- Only one element is changed, and it does not need to be repositioned, so the rest of the array remains unchanged.
- The change in P is:

$$\text{Old contribution: } \bar{a}[i] \cdot v, \quad \text{New contribution: } \bar{a}[i] \cdot (v + 1)$$

- Hence:

$$P \leftarrow P - \bar{a}[i] \cdot v + \bar{a}[i] \cdot (v + 1)$$

• **Conclusion:**

- Only one element changes in the sorted array.
- Its contribution to P changes accordingly.
- We do not need to recompute the full P .
- Correctness is ensured since the sorted order is preserved and the change is applied only at the affected position.

Thus, P is always correctly updated to reflect the new maximum dot product.

Time Complexity

Let n be the size of the arrays and q be the number of update operations.

• **Initial sorting of both arrays:**

Sorting a and b at the beginning takes $\Theta(n \log n)$.

• **Each update operation:**

- Update of value: $\Theta(1)$
- Binary search to find new position: $\Theta(\log n)$
- Updation into new position: $\Theta(1)$
- Dot product update: $\Theta(1)$

So, each update takes $\Theta(\log n)$ time.

• **Total time for q updates:**

$$\Theta(q \log n)$$

Total Time Complexity

$$\boxed{\Theta(n \log n + q \log n)}$$

This shows the algorithm maintains P efficiently after all q updates.

Question 2: Game Score Maximisation

In this problem, we are given an array consisting of n positive integers. We are required to play a game to maximise the score. The game begins with an initial score of 1, and we are provided with k coins, each of which can be used to perform a specific operation that increases the score. With each coin, the following steps are carried out:

1. Choose an interval $[L, R]$ that has not been selected previously.
2. From within the chosen interval, select an element x that has the maximum number of prime factors (counting multiplicity). If more than one such element exists, select the one with the smallest index.
3. Multiply the current score by x .

The objective is to design an algorithm that determines the game's final score. After all k coins have been optimally used so that the score is maximised.

Question (a)

Design an algorithm for determining the final score of the game.

Solution:

We are asked to design an algorithm that computes the maximum final score of the game after spending all k coins according to the given rules. The central challenge is to efficiently decide *which elements* of the array contribute to the final score, and *how many times*.

Algorithm Idea:

Step 1: Prime factor counts. For each element a_i in the array $A[1..n]$, we compute the distinct number of prime factors of a_i . This can be done efficiently using a precomputed *Smallest Prime Factor (SPF)* sieve up to $\max(A)$, which allows each element to be factorised in $O(\log a_i)$ time.

Step 2: Longest intervals with dominance. According to the rules of the game, in each chosen interval $[L, R]$, we must select the element with the highest distinct prime factor count (breaking ties by choosing the smallest index). Hence, each element a_i “dominates” a maximal interval of subarrays where it is the unique choice. We compute these intervals using a monotone stack method:

- L_i is the nearest index to the left of i such that $pf[L_i] > pf[i]$.
- R_i is the nearest index to the right of i such that $pf[R_i] \geq pf[i]$.

This guarantees that in every subarray $[L, R]$ with $L_i < L \leq i \leq R < R_i$, the element a_i is the correct choice.

Step 3: Counting ways. From the dominance intervals, we compute the number of subarrays where a_i is chosen:

$$\text{ways}[i] = (i - L_i) \cdot (R_i - i).$$

This value counts exactly how many distinct intervals $[L, R]$ admit a_i as the winner.

Step 4: Score Maximisation. To maximise the score, we always prefer multiplying by the largest possible array elements. Thus, we prepare a list of pairs $(a_i, \text{ways}[i])$ and sort it in decreasing order of a_i . We then spend our k coins accordingly: for each element, we use it up to $\min(\text{ways}[i], \text{remaining coins})$ times. The score is multiplied accordingly until all k coins are exhausted.

Step 5: Result. After distributing all k coins, the product of all chosen elements is returned as the final score.

Pseudocode:**Algorithm 3: Count_Distinct_Factors****Input:** Integer x , array spf **Output:** Number of *distinct* prime factors of x

```

1  $cnt \leftarrow 0$ ;
2 while  $x > 1$  do
3    $p \leftarrow spf[x]$ ;
4    $cnt \leftarrow cnt + 1$ ;
5   while  $x \bmod p = 0$  do
6      $x \leftarrow x/p$ 
7 return  $cnt$ ;

```

Algorithm 4: Pow**Input:** Base a , exponent $e \geq 0$ **Output:** a^e

```

1  $ans \leftarrow 1$ ;
2 while  $e > 0$  do
3   if  $e \bmod 2 = 1$  then
4      $ans \leftarrow ans \times a$ 
5    $a \leftarrow a \times a$ ;
6    $e \leftarrow \lfloor e/2 \rfloor$ ;
7 return  $ans$ ;

```

Algorithm 5: GameScoreMaximize**Input:** Array $A[1..n]$ of positive integers, number of coins k **Output:** Final maximal game score after k operations

```

1  $M \leftarrow \max(A)$ ;
2  $spf \leftarrow$  Build array of length  $M$  (max element in  $A$ ) storing smallest prime factor of  $i$  in index  $i$  using sieve function;
3 for  $i \leftarrow 1$  to  $n$  do
4    $pf[i] \leftarrow \text{Count\_Distinct\_Factors}(A[i], spf)$ ;
5  $(L[], R[]) \leftarrow$  obtained using a monotonic stack ;
6   store the element with prefix value just smaller (or equal in the case of  $L$ ) on the left and right sides ;
7 for  $i \leftarrow 1$  to  $n$  do
8    $ways[i] \leftarrow (i - L[i]) \times (R[i] - i)$ ;
9  $elems \leftarrow \{(A[i], ways[i]) \mid 1 \leq i \leq n\}$ ;
10 Sort  $elems$  by decreasing first component (value);
11  $score \leftarrow 1$ ;
12  $rem \leftarrow k$ ;
13 foreach  $(val, avail)$  in  $elems$  do
14   if  $rem = 0$  then
15     break;
16    $use \leftarrow \min(avail, rem)$ ;
17    $score \leftarrow score \times \text{Pow}(val, use)$ ;
18    $rem \leftarrow rem - use$ ;
19 return  $score$ ;

```

Question (b)

Give proof of correctness and time complexity analysis of your approach for the part.

Solution:

Proof of Correctness:

We prove that the algorithm correctly computes the maximum score achievable by spending k coins according to the given game rules.

Game Rule Recap. At each step:

- Choose an interval $[L, R]$ not used before.
- Among the elements in $[L, R]$, select the one with the highest number of *distinct prime factors*.
- If there is a tie, the element with the *smallest index* is selected.
- Multiply the current score by the selected element.

We aim to select k such intervals so that the product (score) is maximised.

Step 1 – Prime Factor Count. Each value $A[i]$ is processed using the precomputed SPF (Smallest Prime Factor) array. We divide out each distinct prime factor once and count the number of unique primes that divide $A[i]$. Thus, $pf[i]$ accurately stores the number of distinct prime factors of $A[i]$.

Step 2 – Range Computation. We compute for every index i :

- L_i : the nearest index to the left such that $pf[j] \geq pf[i]$.
- R_i : the nearest index to the right such that $pf[j] > pf[i]$.

Then, for any subarray $[L, R]$ with $L_i < L \leq i \leq R < R_i$, the element $A[i]$ will have the highest number of distinct prime factors and is the *first* such element by index. Thus, according to the problem rules, $A[i]$ is chosen in exactly those subarrays.

Therefore, the number of subarrays where $A[i]$ is the chosen element is:

$$\text{ways}[i] = (i - L_i) \cdot (R_i - i)$$

Which counts all valid combinations of left and right endpoints where $A[i]$ dominates.

Step 3 – Greedy Selection. The final score is obtained by multiplying the selected elements. Since multiplication is commutative and the score is initialised to 1, the objective reduces to maximising the product of k values (with multiplicities) selected from the list:

$$\{A[i] \text{ repeated } \text{ways}[i] \text{ times}\}$$

To achieve this:

- We sort the pairs $(A[i], \text{ways}[i])$ in descending order of $A[i]$.
- We select the largest values first, using up to $\text{ways}[i]$ times or until all k coins are used.

Since larger values contribute more to the product and every coin allows exactly one multiplication, this greedy approach is optimal.

Conclusion. Every step of the algorithm accurately models the game rules:

- It identifies all intervals where each element can be the chosen one.
- It accurately counts the number of such choices.
- It maximises the product by greedily choosing the largest values first.

Hence, the algorithm returns the maximum score that can be achieved using k valid operations.

Time Complexity Analysis:

Let $n = |A|$ be the array size and $M = \max(A)$. We break the algorithm into phases and estimate the cost of each one.

1. Building the SPF (Smallest Prime Factor) table — $\Theta(M \log \log M)$ time, $\Theta(M)$ space. We first set $spf[i] \leftarrow i$ for all $i = 0, \dots, M$, which costs $\Theta(M)$. Then, for each prime $p \leq \sqrt{M}$, we mark its multiples starting from p^2 . Each composite number j may be attempted several times, but the total number of attempts is about

$$\sum_{p \leq \sqrt{M}} \frac{M}{p} \approx M \log \log M.$$

This is the standard sieve analysis, so the preprocessing takes $\Theta(M \log \log M)$ time and $\Theta(M)$ memory.

2. Counting distinct prime factors of $A[i]$ — $\Theta(n \log M)$. Using the SPF table, we can factor any number $x = A[i]$ quickly: each time we divide out one prime $p = spf[x]$, we completely remove all copies of p . Since every division shrinks x by at least a factor of 2, a single x needs at most $\log_2 M$ divisions. Thus, across all n numbers, factorisation costs $\Theta(n \log M)$ operations in total.

3. Computing dominance bounds (L_i, R_i) with monotone stacks — $\Theta(n)$. We do two passes:

- A left-to-right scan maintaining a decreasing stack.
- A right-to-left scan maintaining a non-increasing stack.

Each index is pushed and popped at most once per scan, so the amortised cost is linear. Hence, this step is $\Theta(n)$ time and $\Theta(n)$ space.

4. Computing contributions $ways[i]$ — $\Theta(n)$. Once (L_i, R_i) are known, we just compute

$$ways[i] = (i - L_i) \cdot (R_i - i),$$

This takes constant time per element. So this phase is also linear.

5. Sorting pairs $(A[i], ways[i])$ — $\Theta(n \log n)$. We sort by the values $A[i]$, which dominates the cost here.

6. Greedy multiplication with fast exponentiation — $\Theta(n \log k)$. After sorting, we loop through the pairs. For each value, instead of multiplying val repeatedly, we compute val^{use_i} via fast exponentiation in $\Theta(\log use_i)$. Since $\sum use_i = k$, the total cost sums to at most $\Theta(n \log k)$.

The overall time is

$$T(n, M, k) = \Theta(M \log \log M) + \Theta(n \log M) + \Theta(n \log n) + \Theta(n \log k).$$

Since $\Theta(n \log n)$ dominates when M and k are not exponentially large, the overall running time is

$\Theta(n \log n)$

The memory usage is $\Theta(M)$ for the SPF table plus $\Theta(n)$ for arrays like pf , L , R , and $ways$, i.e. $\Theta(M + n)$.

Question 3: Wealth accumulation

You are given the root of a family tree, represented as a binary tree, where each person (node) except the latest generation has two children: a left child and a right child. Each node contains the initial bank balance of that person and pointers to its children, and there are a total of n nodes

At the end of each year, the following events co-occur for all persons:

1. **Wealth Growth:** The wealth of each person is quadrupled due to bank interest.
2. **Donation:** Each person donates half of their wealth to exactly one of their children.
 - In the first year, the donation goes to the left child.
 - In the second year, the donation goes to the right child.
 - The process alternates between left and right in subsequent years.

Note: The amount of donation for any person each time is fixed for that year before receiving donations from others. Given the initial bank balances, determine the wealth of each person after k years.

Question (a)

Design an algorithm for determining the wealth of each person after k years, where $k \gg n$

Solution:

Algorithm Idea

Let each node be assigned an index, then $W_t \in \mathbb{R}^n$ be the vector that denotes wealth after t years with index i having $W_t[i]$ wealth. Each year is a *linear transformation*:

$$W_{t+1} = A_t \cdot W_t,$$

Where A_t is the update matrix:

- A_{odd} : used when donations go to the left child (odd year).
- A_{even} : used when donations go to the right child (even year).

First, we can create the Arrays $L[1, 2, \dots, n]$ and $R[1, 2, \dots, n]$ by iterating through each node and storing its left and right child for each node in the tree.

Each column i of these matrices has at most two nonzero entries:

$$A_{\text{odd}}(i, i) = 2, \quad A_{\text{odd}}(L(i), i) = 2,$$

$$A_{\text{even}}(i, i) = 2, \quad A_{\text{even}}(R(i), i) = 2,$$

If the corresponding child exists. This represents the fact that every node keeps half of its quadrupled wealth ($2W$) and donates the other half ($2W$) to its child.

Thus, the evolution after k years can be written as:

$$W_k = \begin{cases} (A_{\text{even}} A_{\text{odd}})^{k/2} W_0, & \text{if } k \text{ is even,} \\ A_{\text{odd}} (A_{\text{even}} A_{\text{odd}})^{(k-1)/2} W_0, & \text{if } k \text{ is odd.} \end{cases}$$

Therefore, the problem reduces to efficiently computing a large power of the two-year operator $M = A_{\text{even}} A_{\text{odd}}$ using *exponentiation by squaring*.

Pseudocode:**Algorithm 6:** BUILD_MATRICES($L[1..n], R[1..n]$)**Input:** Child arrays $L[i], R[i]$ (0 if missing)**Output:** $A_{\text{odd}}, A_{\text{even}}$

```

1 Initialize  $A_{\text{odd}}, A_{\text{even}} \leftarrow 0_{n \times n}$ ;
2 for  $i \leftarrow 1$  to  $n$  do
3    $A_{\text{odd}}[i, i] \leftarrow 2$ ;
4    $A_{\text{even}}[i, i] \leftarrow 2$ ;
5   if  $L[i] \neq 0$  then
6      $A_{\text{odd}}[L[i], i] \leftarrow 2$ 
7   if  $R[i] \neq 0$  then
8      $A_{\text{even}}[R[i], i] \leftarrow 2$ 
9 return  $A_{\text{odd}}, A_{\text{even}}$ 

```

Algorithm 7: MATRIX_POWER(A, e)**Input:** $A \in \mathbb{R}^{n \times n}$, integer $e \geq 0$ **Output:** A^e

```

1  $P \leftarrow I_n, B \leftarrow A$ ;
2 while  $e > 0$  do
3   if  $e$  odd then
4      $P \leftarrow P \cdot B$ 
5    $B \leftarrow B \cdot B$ ;
6    $e \leftarrow \lfloor e/2 \rfloor$ ;
7 return  $P$ 

```

Algorithm 8: K_TH_YEAR_WEALTH(L, R, W_0, k)**Input:** Binary Tree Node**Output:** W_k

```

1  $L[1, 2, \dots, n], R[1, 2, \dots, n] \leftarrow$  We will make these arrays using DFS/BFS from the root of the binary tree;
2  $(A_{\text{odd}}, A_{\text{even}}) \leftarrow \text{BUILD\_MATRICES}(L, R)$ ;
3  $M \leftarrow A_{\text{even}} \cdot A_{\text{odd}}$ ;
4  $P \leftarrow \text{MATRIX\_POWER}(M, \lfloor k/2 \rfloor)$ ;
5 if  $k$  even then
6    $W_k \leftarrow P \cdot W_0$ ;
7 else
8    $W_k \leftarrow A_{\text{odd}} \cdot P \cdot W_0$ ;
9 return  $W_k$ 

```

Question (b)

Give a time complexity analysis of your approach for part (a).

Solution:

Time Complexity

The algorithm reduces the k -year wealth evolution to a matrix power:

$$W_k = \begin{cases} (A_{\text{even}}A_{\text{odd}})^{k/2}W_0, & k \text{ even,} \\ A_{\text{odd}} (A_{\text{even}}A_{\text{odd}})^{\lfloor k/2 \rfloor}W_0, & k \text{ odd,} \end{cases} \quad M = A_{\text{even}}A_{\text{odd}}.$$

Step 1: Building L and R . A single BFS/DFS traversal assigns indices $1..n$ to nodes and records their left and right children. Each node is visited once and at most two entries are written, so this step runs in $\Theta(n)$ time and uses $\Theta(n)$ space.

Step 2: Constructing operators. Building A_{odd} and A_{even} requires scanning all n nodes once, inserting at most two nonzero entries per column. This takes $\Theta(n)$ time and $\Theta(n^2)$ space to store in dense form.

Step 3: Powering the matrix. We must compute $M^{\lfloor k/2 \rfloor}$. Naively multiplying M repeatedly would take $\Theta(kn^3)$ time, which is infeasible for large k . Exponentiation by squaring reduces the number of multiplications to $\Theta(\log k)$ by exploiting the binary expansion of the exponent.

$$\text{Cost} = \Theta(n^3) \text{ per multiply} \times \Theta(\log k) \text{ multiplies} = \Theta(n^3 \log k).$$

Step 4: Final multiplications. Once the power is computed, we multiply it by the initial vector (and by A_{odd} if k is odd). These take $\Theta(n^2)$ and are strictly smaller than $\Theta(n^3 \log k)$, so they do not affect the asymptotic bound.

Overall. The dominating term is from exponentiation:

$$\boxed{\text{Total running time} = \Theta(n^3 \log k), \quad \text{space} = \Theta(n^2).}$$

This is exponentially faster in k than direct simulation ($\Theta(nk)$), and becomes preferable whenever $k \gg n$.

Question 4. The King's Punishment

In this problem, we are given that King Robert wants to inspect the heights of his Knights, each having a unique height. The Knights are standing in a line and will be punished if they are standing such that the rule is broken. The rule is

- A knight is punished once for every taller knight standing before him in the lineup.

Question (a)

Describe a brute force algorithm to compute the punishments received by each knight. What would be its time complexity?

Solution:

Brute Force Algorithm Idea

In this question, we need to compute the number of punishments received by each Knight by the king, which is the number of Knights before that Knight who are taller than the particular Knight.

The first idea that comes to mind for this part is that for each Knight, we calculate the number of Knights that have a height taller than the particular Knight standing before him by iterating through the sequence each time for every Knight.

For each Knight i , we scan all the knights $j < i$ and count how many of them have height $h[j] > h[i]$. Store this in an array Punishment with i index storing the number of Knights having the above property.

Pseudocode:

Algorithm 9: Punishment_Count

Input: Array $H[1..n]$ of numbers each $H[i]$ storing height of Knight i

Output: Punishment received by each Knight

```

1 punishment  $\leftarrow$  array of size  $n$ ;
2 for  $i \leftarrow 1$  to  $n$  do
3   punishment[ $i$ ]  $\leftarrow$  0;
4   for  $j \leftarrow 1$  to  $i - 1$  do
5     if  $H[i] < H[j]$  then
6       punishment[ $i$ ]  $\leftarrow$  punishment[ $i$ ] + 1
7 return punishment;

```

Time and Space Complexity

Time Complexity:

The pseudocode has two nested loops:

- Outer loop runs n times (for each Knight $i = 1 \dots n$)
- Inner loop runs up to $i - 1$ times for each Knight i

So total iterations:

- $0 + 1 + 2 \dots + n - 1 = \frac{n(n-1)}{2}$

Hence,

$$T(n) = \Theta(n^2)$$

Space Complexity: The output array requires $\Theta(n)$ space, and only $\Theta(1)$ extra variable space is used for variables. Hence,

$$S(n) = \Theta(n)$$

Question (b)

Suppose we insert the knights one by one in the order they stand. Can we maintain their heights in a data structure that allows us to quickly compute the punishments for the newly added knight? Describe the required operations of such a data structure and write the pseudocode for them.

Solution:

Data Structure:

Maintain a Binary Search Tree (BST) keyed by distinct heights. Each node u stores three integers:

$$\text{key}(u), \quad \text{lsize}(u) = |\text{nodes in } u.\text{left subtree}|, \quad \text{rsize}(u) = |\text{nodes in } u.\text{right subtree}|.$$

Required operations and rules for counting punishments Knights arrive in the standing order with heights $h_1, \dots, h_n$. When a new height h arrives, we descend from the root to its insertion point. At a node with key k :

- If $h < k$: this node and *all nodes in its right subtree* are taller than h . Add $1 + \text{rsize}$ of the current node to the running count and move to the left child. Since the new node will be placed in the left subtree of this node, increment its lsize by 1.
- If $h > k$: no contribution; move to the right child and increment rsize of this node by 1.

At the null position, create a node with ($\text{key} = h$, $\text{lsize} = 0$, $\text{rsize} = 0$). The accumulated sum along the search path is the number of punishments for h .

Pseudocode:

Algorithm 10: InsertAndCount(T, h)

Input: Root T of the BST; new height h

Output: Punishments counted for h and updated T

```

1 curr ← T, parent ← NIL, ans ← 0;
2 while curr ≠ NIL do
3   parent ← curr;
4   if h < curr.key then
5     ans ← ans + 1 + curr.rsize;
6     curr.lsize ← curr.lsize + 1;
7     curr ← curr.left;
8   else
9     curr.rsize ← curr.rsize + 1;
10    curr ← curr.right;
11 node ← new node with (key = h, lsize = 0, rsize = 0);
12 if parent = NIL then
13   T ← node;
14 else if h < parent.key then
15   parent.left ← node;
16 else
17   parent.right ← node;
18 return ans;
```

Algorithm 11: Punishments

Input: Array of distinct heights $h[1..n]$ in standing order

Output: Array $p[1..n]$ of punishments

```

1 T ← NIL;
2 for i ← 1 to n do
3   p[i] ← InsertAndCount(T, h[i]);
4 return p;
```

Question (c)

Do you need to do something additional to ensure these operations remain efficient as the number of knights grows? (Think about what happens if the data structure becomes unbalanced.)

Solution:

In this solution, we can face problems if we keep adding heights; then the height of BST can grow to become $\Theta(n)$ for some cases in heights like sorted heights. In that case, both updation and insertion degrade and take $\Theta(n)$ time, and the overall running time becomes $\Theta(n^2)$.

To ensure that operations remain efficient as the number of Knights increases, we should balance the tree again and again after some steps (in order of the number of nodes), so that the operations take their usual time and do not degrade. So **rebalancing** is one of the steps through which we can make the algorithm efficient for all inputs.

Other ways can be to use self-balancing trees like 2-3 Trees and Red-Black Trees, in which for all the inputs, the height of the tree remains $\log(n)$.

One other way that we can do such that the height remains $\log(n)$ is using randomised insertion, which means inserting the heights in random order, which gives an average $\Theta(\log(n))$ in insertion and updation of the tree, but does not protect the tree from adversarial orders like a sorted array.

Question (d)

Analyse the time complexity of both the update and query operations in your data structure. Hence, give the overall time complexity of your algorithm.

Solution:

Time Complexity

We process knights using the BST augmented with two counters per node ($lsize, rsize$).

Per-operation costs. Let $h(T)$ denote the height of the current BST .

- **Query during insertion (punishment count).** Along the root-to-leaf search path, each left turn contributes $1 + rsize(\cdot)$ in $\Theta(1)$ time. Hence the query cost is $\Theta(h(T))$.
- **Update (insert).** The insertion follows the same path and updates one of $lsize/rsiz$ at each visited node in $\Theta(1)$; thus the update cost is also $\Theta(h(T))$.

Overall complexity. Let n be the number of knights.

- **Balanced BST (guaranteed logarithmic height).** With $h(T) = \Theta(\log n)$ throughout, each knight costs $O(\log n)$, giving total

$$T(n) = \Theta(n \log n), \quad S(n) = \Theta(n).$$

- **Unbalanced (basic) BST .** In the worst case (e.g., sorted input) $h(T) = \Theta(n)$, so the i -th insertion costs $\Theta(i)$ and

$$T(n) = \Theta(1 + 2 + \dots + (n-1)) = \Theta(n^2), \quad S(n) = \Theta(n).$$

So the overall Time Complexity depends on the height and the complexity is $\Theta(n * h(T))$ where $h(T)$ is the height of the tree after all the insertions. If we take precautionary measures as said in Question(c), the height of the tree will be in order of $\log(n)$, so overall complexity will be $\Theta(n \log(n))$; else it will be $\Theta(n^2)$.